Kit do Desenvolvedor

Vai e Volta

Padrões e Fluxo de Trabalho

Disciplina *Projeto e Desenvolvimento de Software II* — FURG

Campo	Valor
Versão	1.1
Data	10/10/2026
Gestão	Lincon Avila, Miguel Casarin, Pietro Noble e Lucas Derick
Responsáveis pelo documento	**Lincon Avila** (Tech Lead), **Miguel Casarin** (Product Owner) e **Lucas Derick** (Scrum Master)
Público	Equipe de desenvolvimento (4 pessoas)
Documento complementar	Documento Técnico do Projeto, versão 1.0 (escopo, regras, arquitetura, tecnologias, estrutura de pastas, testes e CI)
Entrega da disciplina	**MVP**

Sumário

1 Visão geral **2**

2 Quem faz o quê **2**

3 Onde encontrar as coisas **2**

3.1 Identificadores . . . 3

3.2 Como uma tarefa aparece no SilverBullet . . . 3

3.3 O que vocês podem editar . . . 3

4 Sprints e cerimônias **4**

5 Tarefas **5**

5.1 Estados (campo Status) . . . 5

5.2 Regras . . . 5

6 Definition of Ready e Definition of Done **5**

6.1 Definition of Ready (história pode entrar na sprint) . . . 5

6.2 Definition of Done . . . 5

7 Git e GitHub **6**

7.1 Branches . . . 6

7.2 Commits (Conventional Commits) . . . 6

7.3 Pull Requests . . . 7

7.4 Proteções da `main` . . . 7

8 Fluxo completo de uma tarefa **7**

9 Revisão de código e aceite **8**

10 CI: o que o repositório precisa ter **8**

11 Bugs, impedimentos e mudanças **9**

11.1 Bugs . . . 9

11.2 Impedimentos . . . 9

11.3 Mudanças de escopo . . . 10

12 Comunicação, releases e boas práticas **10**

12.1 Comunicação . . . 10

12.2 Releases e documentação . . . 10

12.3 Boas práticas e segurança . . . 10

13 Checklists e modelos **11**

13.1 Antes de começar uma tarefa . . . 11

13.2 Antes de marcar o PR como “Ready” . . . 11

13.3 Fim do dia . . . 11

13.4 Template de Pull Request (GitHub) . . . 11

13.5 Mensagem de impedimento (WhatsApp) . . . 12

Histórico de revisões **13**

1 Visão geral

Este kit é o guia de trabalho da **equipe de desenvolvimento**, entregue pela equipe de gestão. Ele diz **como** trabalhamos: papéis, sprints, tarefas, Git, revisão, CI, bugs e comunicação. O **que** construir e **como o projeto é estruturado** está no **Documento Técnico do Projeto**. Em conflito, o documento técnico vale para decisões técnicas; este kit vale para decisões de processo. Os dois devem ser lidos em conjunto.

Itens entre colchetes, como [definir], serão preenchidos pela gestão.

Em 1 minuto

- Trabalhamos em **Scrum adaptado**, com sprints de **7 dias corridos (sábado a sexta)**.
- A **gestão** define **o quê e por quê**: backlog, prioridades, critérios de aceite e aceite final.
- **Vocês** decidem **como e quem**: soluções técnicas, divisão de tarefas e estimativas.
- O planejamento vive no **SilverBullet**. O código vive no **GitHub**. O **ID da tarefa** (`H012-1`) liga os dois.
- **Nada entra na `main` sem PR, CI verde, revisão de um colega dev e aceite do QA.**
- A entrega é um **MVP**: prazo fixo, escopo negociável. Se apertar, cortamos funcionalidade, **nunca** a Definition of Done.

2 Quem faz o quê

Papel	Pessoa	Procure quando…
Product Owner (PO)	Miguel Casarin	Dúvida de requisito, prioridade ou critério de aceite
Scrum Master (SM)	Lucas Derick	Impedimento, dúvida de processo ou de SilverBullet
Tech Lead	Lincon Avila	Repositório, PR, CI, release, decisões técnicas e alterações no documento técnico
QA	Pietro Noble	Teste de aceite e bugs

Vocês: estimam, assumem e concluem as tarefas; **revisam o código uns dos outros**; mantêm o SilverBullet atualizado; sinalizam impedimentos **no mesmo dia**; atualizam a documentação afetada por suas mudanças. A gestão não atribui tarefas individualmente nem dita soluções técnicas.

3 Onde encontrar as coisas

Preciso de…	Onde
Quadro da sprint, tarefas bloqueadas, aguardando aceite	**SilverBullet** → projeto "vai e volta" → `Atividades` → aba **Kanban**
Ler uma história e seus critérios de aceite	**SilverBullet** → Documentação de Requisitos e Critérios de Aceitação
Minhas tarefas	**Atividades** → `Tabela`, na busca digite `Sprint NN` ou o ID da história (ex.: `H012`)
Bugs e bloqueios	**SilverBullet** → `Questões`
Meta da sprint e atas	Atividade `Ata Sprint NN` (na busca, digite `Ata`)

Preciso de…	Onde
Decisões já tomadas	**SilverBullet** → `Mudanças`
Escopo, funcionalidades (`F-nn`), regras de negócio (`RN-nn`), arquitetura, tecnologias, API, testes e CI	**Documento Técnico do Projeto**
O que está **fora do MVP**	**Documento técnico** → seção "Fora do MVP"
Código, PRs, pipeline	**GitHub**

3.1 Identificadores

Tipo	Formato	Exemplo
História	`H-NNN`	`H-012`
Tarefa	`HNNN-N` (história + sequência)	`H012-3`
Bug	`B-NNN`	`B-007`
Funcionalidade	`F-nn` (documento técnico)	`F-34`
Regra de negócio	`RN-nn` (documento técnico)	`RN-42`
Caso de teste	`<NÍVEL>-<ÁREA>-<nn>`; os IDs das regras são reservados no documento técnico	`API-RNT-04`

IDs nunca são reaproveitados. **Toda branch, commit e PR cita o ID da tarefa** (ou do bug). Quando a tarefa implementa ou altera uma regra, cite também as `RN-nn` e `F-nn` no PR, e coloque o ID do caso de teste no nome da função ou do arquivo de teste (por exemplo, `test_API_RNT_04_...`).

3.2 Como uma tarefa aparece no SilverBullet

Tarefa é uma **Atividade** no SilverBullet (`Atividades` → `Nova Atividade`), preenchida assim:

- **Nome:** ID + até 3 palavras, **máximo 30 caracteres**. Ex.: `H012-1 Cadastro usuário`.
- **Descrição:** 1ª linha `Sprint NN`; 2ª linha `PR: <link>`; 3ª linha só quando houver `[QA]` ou `[BLOQUEADO]`; depois, o que a tarefa faz. A 1ª linha é o que permite achar a sprint na busca.
- **Responsáveis:** uma pessoa.
- **Fase:** `Sprint NN`.
- **Requisitos Atrelados:** a história (ex.: `H-012`).
- **Dependência entre Atividades:** a tarefa que precisa terminar antes, se houver.
- **Datas estimadas e Duração Estimada em Horas:** preencham no Planning; a **Duração Realizada** é preenchida ao concluir.
- **Atividade de Desenvolvimento?: Sim** para tarefas de código (as de gestão ficam com Não).
- **Status:** conforme a seção 5.

Vocês criam as tarefas no Planning e mantêm Status, Responsáveis e a Descrição (sprint e link do PR) atualizados.

3.3 O que vocês podem editar

Conteúdo	Quem edita
Histórias, critérios de aceite, prioridade, meta da sprint	PO (vocês leem; dúvidas vão ao PO ou ao Planning)
Status, Responsáveis, Fase, datas, horas e Descrição (sprint e link do PR) das **suas** atividades	**Vocês**
Resultado de aceite e bugs	QA (vocês registram o PR da correção na Solução Adotada da Questão)
Atas, riscos, métricas	SM

Regras de convivência:

1. Mexa só no que é seu; dúvida vira mensagem no canal.
2. **Nunca apague nada.** Item cancelado volta para Backlog com `[CANCELADO: motivo]` na Descrição.
3. Mudou o estado? **Atualize na hora.**
4. Evite editar o mesmo item ao mesmo tempo que outra pessoa; combine no canal.

4 Sprints e cerimônias

Item	Definição
Duração	**7 dias corridos** (sábado a sexta)
Sprints finais	**Estabilização e entrega do MVP**: bugs, regressão e documentação. **Sem funcionalidades novas** (*feature freeze*). Antes da apresentação: `make backup` e **teste de restauração** (`make restore`), conforme o documento técnico
Calendário	Sprint 0: 10/10 a 16/10; Sprint 1: 17/10 a 23/10; depois, de 7 em 7 dias (SilverBullet → Plano de Geren. do Cronograma)

Cerimônia	Quando	Duração	Objetivo
Planning (inclui refinamento das histórias)	Sábado, início da sprint	até 1h	Meta da sprint; vocês estimam as histórias e criam as tarefas
Daily (assíncrona, no grupo de WhatsApp com todos)	Segunda, quarta e sexta, até as 12h	15 min	O que fiz, o que farei, impedimentos
Review + Retro	Sábado, fim da sprint	até 1h	Demonstrar o entregue (o PO aceita as histórias) e definir 1 a 3 melhorias

Planning e Review + Retro acontecem na mesma reunião de sábado; o SM avisa o horário no grupo até a sexta anterior.

Estimativa: pontos (1, 2, 3, 5, 8) **por história**, por consenso ou *Planning Poker*. Acima de **8 pontos**, quebrar a história. Tarefas não precisam de pontos: devem ser de **uma pessoa** e de **até cerca de 1 dia**.

5 Tarefas

5.1 Estados (campo Status)

Status	Significado	Quem muda
Backlog	Fora da sprint (ou cancelada, com `[CANCELADO: motivo]`)	PO / Vocês
A fazer	Na sprint, sem início	Vocês
Em andamento	Alguém trabalhando	Dev responsável
Em revisão	PR aberto, aguardando revisão de colega dev **ou** aceite do QA	Dev
Concluído	Aceito pelo QA e mergeado	QA / Tech Lead
Bloqueado	Abra uma **Questão** e escreva `[BLOQUEADO]` na 3ª linha da Descrição da atividade (com o motivo)	Quem identificar

Distinção dentro de Em revisão: depois da aprovação do colega, escreva `[QA]` na 3ª linha da Descrição; o QA coloca o *label* `aceito` no PR.

5.2 Regras

1. Cada tarefa tem **um único responsável**.
2. **No máximo 2 tarefas em Em andamento por pessoa.** Termine antes de começar outra.
3. Tarefa em Em revisão parada há mais de **24h úteis** vai para a Daily.
4. Prefira tarefas **pequenas e independentes** e revezem as tarefas, para ninguém ser dono exclusivo de uma parte.

6 Definition of Ready e Definition of Done

6.1 Definition of Ready (história pode entrar na sprint)

- ☐ Formato "Como... quero... para..." e critérios de aceite claros.
- ☐ Estimada por vocês (até 8 pontos).
- ☐ Dúvidas respondidas e dependências conhecidas.
- ☐ Prioridade definida pelo PO.

> História fora do DoR? **Peçam ajuste no Planning** em vez de começar assim mesmo.

6.2 Definition of Done

Sua parte (tarefa/PR)

- ☐ Faz o que a história e os critérios de aceite pedem, sem extras não solicitados.
- ☐ Testes escritos ou atualizados e passando; nada que já funcionava foi quebrado. Toda regra de negócio (`RN-nn`) tocada tem ao menos um teste com o ID reservado; cobertura de linhas do backend em `services` e `routers` de pelo menos 70%.
- ☐ `make lint` e `make test` passando localmente; CI **verde**.
- ☐ Documentação afetada atualizada **no mesmo PR**: `README`, documento técnico, `docs/openapi.json` (`make openapi`; o CI falha se estiver desatualizado) e `.env.example` (em sincronia com a

tabela de variáveis).

- [ ] Mudança de schema feita por **migração Alembic**, compatível com a versão anterior da API e testada sobre banco vazio.
- [ ] Sem segredos, senhas ou dados pessoais, nem nos logs (nunca registrar senha, CPF, e-mail, telefone, endereço, token ou corpo de requisição).
- [ ] PR preenchido pelo template e **aprovado por 1 colega dev**.
- [ ] O projeto roda pelas instruções do repositório (`make up`, só com Docker instalado), **sem passos manuais extras**.

Parte da gestão (história)

- [ ] QA validou os critérios de aceite.
- [ ] PO deu o aceite na Review.

> **Regra de ouro:** se não cumpre a DoD, **não está pronto** e não conta como entregue nem na velocidade da sprint.

7 Git e GitHub

7.1 Branches

- `main` é **protegida**: só entra código por PR.
- Tipos: `feat/`, `fix/`, `docs/`, `refactor/`, `test/`, `chore/`.
- Nome: `<tipo>/<id-da-tarefa>-<descricao-curta>`

```
feat/H012-1-cadastro-de-usuario
fix/B-007-erro-ao-validar-datas
```

- Minúsculas e hífens, sem acento (o ID pode ter maiúscula); descrição de até cerca de 5 palavras.
- **Uma branch por tarefa**, vida curta (ideal até 3 dias), **excluída após o merge**.
- Atualize a branch com a `main` antes de finalizar o PR.
- Correção urgente usa `fix/` e pede revisão acelerada no canal.

7.2 Commits (Conventional Commits)

```
<tipo>: <descrição no imperativo, minúscula, sem ponto>

Refs H012-1
```

Tipos: `feat`, `fix`, `docs`, `refactor`, `test`, `chore`. Exemplo:

```
feat: adicionar filtro por período

Refs H012-1
```

- Commits **pequenos** (uma mudança lógica por commit).
- Idioma das mensagens de commit e dos títulos de PR: **português**; o tipo (`feat`, `fix`...) permanece em inglês. Os identificadores técnicos do código (tabelas, colunas, rotas, variáveis e funções) são sempre em inglês; só o texto exibido ao usuário fica em português, conforme o documento técnico.

- **Proibido:** mensagens vazias (`ajustes`, `update`, `teste`) e `push --force` em branch compartilhada ou na `main`.

7.3 Pull Requests

- Todo código entra na `main` **somente por PR**.
- Título com o ID: `feat(H012-1): adicionar cadastro de usuário`.
- Abra como **Draft**; marque **"Ready for review"** só ao cumprir a DoD.
- Preencha o **template de PR** (seção 13.4) e registre o link na 2ª linha da Descrição da atividade (`PR: <link>`).
- PR pequeno (referência: cerca de 400 linhas); maior que isso, divida.
- Merge por **Squash and merge**, feito pelo **Tech Lead** depois do aceite do QA.

7.4 Proteções da `main`

Já configuradas pelo Tech Lead: PR obrigatório com **1 aprovação** de outro dev, CI verde (os *required status checks* são os jobs listados na seção 10), branch atualizada, conversas resolvidas, push direto e *force push* bloqueados. O aceite do QA é registrado no PR (comentário de aceite **e** *label* `aceito`) e o Tech Lead só faz o merge com ele.

Contas: vocês têm permissão de escrita no repositório. **2FA obrigatório** em todas as contas do GitHub.

8 Fluxo completo de uma tarefa

```
1. Planning: a história entra na sprint e vocês criam as tarefas
2. Você assume a tarefa → Status Em andamento
3. Cria a branch tipo/H012-1-descricao a partir da main atualizada
4. Implementa, com commits pequenos
5. Abre o PR (Draft → Ready); a CI roda → Status Em revisão
6. CI vermelha? Corrija.  CI verde? Um colega dev revisa e aprova → escreva [QA] na Descrição (segue em Em
↪  revisão)
7. QA testa os critérios de aceite
   ├─ Reprovou: o QA registra o que falhou; a tarefa volta a Em andamento (passo 4)
   └─ Aprovou: o Tech Lead faz o merge (squash) → Status Concluído
8. Review da sprint: o PO aceita a história
```

Resultados do QA: `Aprovado`, `Aprovado com ressalvas` (vira bug menor) ou `Reprovado` (volta para vocês com os critérios que falharam).

Se o PO reprovar uma história na Review: ela volta ao backlog e vira bug ou nova tarefa descrevendo o ajuste.

Prazos

Etapa	Prazo
Revisão de um colega dev	1 dia útil
Aceite do QA	2 dias úteis após o PR estar pronto

Se a gestão estourar o prazo, **sinalizem na Daily**; o SM escala.

9 Revisão de código e aceite

Checklist do revisor técnico (colega dev)

- ☐ Faz o que a história pede, sem extras?
- ☐ Segue os padrões (documento técnico e este kit)?
- ☐ Legível, sem duplicação evidente, nomes claros?
- ☐ Trata erros e casos-limite?
- ☐ Sem segredos, dados sensíveis ou código morto?
- ☐ Testes e documentação adequados (regras com teste e ID reservado, `openapi.json` e `.env.example` atualizados)?
- ☐ Regras no lugar certo (regras de negócio em `services`, não em `routers`) e dados privados saindo só pelos schemas públicos e privados (RN-15, RN-17, RN-35)?
- ☐ Datas e "hoje" usam `today_br()`, nunca `date.today()`?
- ☐ O título do PR e os commits seguem o padrão?

O que o QA verifica no aceite

- ☐ O projeto roda pelas instruções do repositório, sem passos extras (checklist de verificação do ambiente, no documento técnico)?
- ☐ Todos os critérios de aceite passaram, incluindo os de erro?
- ☐ O que já existia segue funcionando?
- ☐ Os fluxos afetados continuam corretos nos roteiros de ponta a ponta do documento técnico?
- ☐ Há evidência anexada ao PR?

> **Dica:** antes de pedir aceite, **percorram vocês mesmos os critérios de aceite da história**. É o que mais evita reprovação.

Cultura de revisão: comentar o código, nunca a pessoa; diferenciar **bloqueante** de **sugestão** (prefixo `nit:`); explicar o porquê; reconhecer boas soluções.

10 CI: o que o repositório precisa ter

A verificação automática usa **GitHub Actions**, em um único workflow (`.github/workflows/ci.yml`), executado em PRs e em pushes na `main`. O documento técnico (seção "Integração contínua") define **como** ele é implementado; a gestão exige **o que** deve existir:

Verificação em cada PR	Job (*required status check*)	Bloqueia merge?
Linter e formatação	`lint-backend`, `lint-frontend`	Sim
Testes automatizados	`test-backend` (inclui a conferência do `docs/openapi.json`), `test-frontend`	Sim
Build e compilação	`build-frontend`, `build-images` (sem publicar)	Sim
Segredos expostos	`secrets-scan` (Gitleaks)	Sim

Os testes de ponta a ponta (Playwright) rodam em job separado (`e2e`), na `main` e à noite, **fora** dos checks obrigatórios.

Regras:

- **CI vermelha = PR não é revisado nem mergeado.** Quem quebrou a pipeline corrige primeiro.

- Os mesmos comandos rodam local e no CI: `make lint` e `make test`. Rode-os antes de abrir o PR.
- Renomear um job exige atualizar a proteção da `main` (Tech Lead).
- Segredos nunca ficam no repositório: o `.env` real é ignorado pelo Git e só o `.env.example` (valores falsos) é versionado. O CI não usa segredos de repositório; apenas valores de teste definidos no próprio workflow.
- Mudanças na pipeline passam pelo **Tech Lead**.
- Configurar a pipeline é tarefa como qualquer outra, feita na Sprint 0 ou na primeira sprint.
- O **Dependabot** (pip/uv, npm, Docker e GitHub Actions) abre PRs semanais; atualização de série maior só em PR dedicado.
- Não há deploy nem publicação de imagens: o projeto roda localmente via Docker.

11 Bugs, impedimentos e mudanças

11.1 Bugs

Bug é uma **Questão** no SilverBullet (`Questões` → `Nova Questão`). O formulário não tem campo de nome, então o ID vai na 1ª linha da Descrição, a gravidade em **Prioridade** e o PR da correção em **Solução Adotada**:

```
Descrição e Potencial Impacto: B-007 Erro validar datas. <o que acontece>
Prioridade: Alta (ver tabela abaixo)
Solução Adotada: PR: <link> (preencher ao corrigir)
```

Prioridade (gravidade)	Definição	Tratamento
Crítico (**Alta**)	Sistema inutilizável, perda de dados ou fluxo principal parado	Interrompe a sprint; correção imediata
Importante (**Média**)	Funcionalidade importante quebrada, sem contorno	Corrigir na sprint atual
Menor (**Baixa**)	Tem contorno ou é cosmético	Backlog, quando houver folga

- QA e PO definem a severidade.
- Estados do bug: aberto → corrigido (vocês escrevem o PR em Solução Adotada e avisam o QA) → fechado (o QA, após revalidar, anota em Comentário | Histórico). Vocês escolhem quem corrige.
- **Nada de correção "por fora"**: Questão registrada → branch `fix/` → PR. Para fechar, o QA valida de novo.
- Encontrou um bug? Avise o QA ou registre uma Questão; **não corrija em silêncio**.

11.2 Impedimentos

1. Abra uma **Questão** e escreva `[BLOQUEADO]` na 3ª linha da Descrição da atividade, com **o motivo** e **quem pode resolver**.
2. Avise **no mesmo dia** no grupo do WhatsApp ou na Daily (modelo na seção 13.5).
3. O **SM** acompanha até desbloquear. Sem solução em **2 dias úteis**, ele escala ao professor ou orientador.

11.3 Mudanças de escopo

- Vocês **não alteram escopo por conta própria**, nem adicionam funcionalidades "já que estão ali".
- Toda proposta de mudança vai ao **PO**, por escrito.
- No MVP, **aumento de escopo vai para o pós-MVP**, salvo se o PO justificar que é indispensável e houver troca equivalente de esforço.
- Os itens da seção "Fora do MVP" do documento técnico **não devem ser implementados nem assumidos como existentes**.
- Mudança em decisão técnica (stack, regras, contrato da API, modelo de dados) é alinhada com o **Tech Lead** e refletida no documento técnico no mesmo PR.

12 Comunicação, releases e boas práticas

12.1 Comunicação

Canal	Uso
WhatsApp — grupo com a gestão e os devs	Dailies, avisos e dúvidas rápidas (não substitui registro formal)
SilverBullet	Fonte oficial do planejamento
GitHub (PRs)	Fonte oficial sobre o código e o aceite

1. Decisão tomada em conversa informal é **registrada** em `Mudanças`, na ata ou no PR.
2. Dúvida de requisito: **pergunte ao PO**; a resposta fica na página da história.
3. Respostas em até **24h** (mensagens) e **1 dia útil** (menções em PR).
4. Ausência prevista: avise o SM com **24h** de antecedência.
5. Respeito e linguagem profissional em todos os canais.

12.2 Releases e documentação

- Versões `0.x.y` durante o desenvolvimento (uma por sprint aceita); **o MVP é `1.0.0`**.
- Ao fim de cada sprint aceita, o **Tech Lead** cria a tag (`vX.Y.Z`) e a release no GitHub, com notas geradas a partir dos PRs mergeados. Por isso títulos de PR e commits no padrão importam. A release não publica imagens nem pacotes; a demonstração sobe a partir do checkout da tag.
- `README` e documentação técnica (incluindo `docs/openapi.json` e `.env.example`): **vocês** atualizam no mesmo PR da mudança.

12.3 Boas práticas e segurança

- **Nunca** commitar senhas, chaves ou tokens. Mantenha o `.env.example` sem valores reais e o `.gitignore` adequado (`.env` e `backups/` ficam fora do Git). Segredos nunca entram em `ARG`, `ENV` ou camadas de imagem Docker.
- Vazou um segredo? Considere-o **comprometido**, troque imediatamente (procedimento de rotação no documento técnico) e **avise o SM**.
- Dados pessoais (LGPD): CPF, telefone e endereço só saem da API conforme as regras de visibilidade; **nunca** em logs, commits, *prints* de PR ou dados de exemplo reais.
- Dados de exemplo: **somente fictícios** (`make seed`, só em desenvolvimento e demonstração).
- Respeite licenças e atribua código de terceiros.

- **IA generativa** (se permitida pela disciplina): o código gerado deve ser **entendido e revisado** por quem o submeter; o autor do PR responde por ele.

13 Checklists e modelos

13.1 Antes de começar uma tarefa

- [ ] Entendi os critérios de aceite da história?
- [ ] Sou o responsável e a atividade está em Em andamento (máximo 2 comigo)?
- [ ] Criei a branch `tipo/H0xx-N-descricao` a partir da `main` atualizada?

13.2 Antes de marcar o PR como "Ready"

- [ ] Commits e título no padrão, citando o ID
- [ ] `make lint` e `make test` passando; CI verde
- [ ] Regras (`RN-nn`) tocadas têm teste com ID reservado
- [ ] Documentação atualizada (`README`, documento técnico, `make openapi`, `.env.example`)
- [ ] Migração Alembic incluída, se houve mudança de schema
- [ ] Sem segredos ou dados pessoais (inclusive em logs)
- [ ] Percorri os critérios de aceite da história
- [ ] Template preenchido; link do PR na 2ª linha da Descrição
- [ ] Sem conflitos com a `main`

13.3 Fim do dia

- [ ] Status das minhas tarefas atualizado
- [ ] Impedimentos sinalizados
- [ ] PRs de colegas pendentes olhados

13.4 Template de Pull Request (GitHub)

```
## Tarefa
H0xx-N (link da atividade no SilverBullet)
Regras e funcionalidades: RN-nn, F-nn (quando relevante)

## Descrição
<!-- O que foi feito e por quê -->

## Como validar
<!-- Passos e evidências (prints, saída de testes) -->

## Checklist (DoD)
- [ ] Padrão de branch e commits
- [ ] Testes passando (make test); nada anterior quebrou; regras com teste (ID reservado)
- [ ] Lint passando (make lint)
- [ ] Documentação atualizada (inclui openapi.json e .env.example)
- [ ] Migração Alembic, se houve mudança de schema
- [ ] Sem segredos ou dados pessoais (inclusive em logs)
- [ ] CI verde
- [ ] Revisado por 1 colega dev

## Para a gestão
- [ ] Critérios de aceite validados (QA)
```

13.5 Mensagem de impedimento (WhatsApp)

```
[BLOQUEIO] H012-1 — <o que está travado>
Motivo:
Quem pode resolver:
Impacto/prazo:
```

Histórico de revisões

Versão	Data	Descrição
1.0	10/10/2026	Versão inicial
1.1	10/10/2026	Adaptação ao SilverBullet em uso (Atividades, Kanban, Questões, Mudanças); sprint de 7 dias; cerimônias aos sábados; Daily no WhatsApp; idioma dos commits; SLAs definidos.